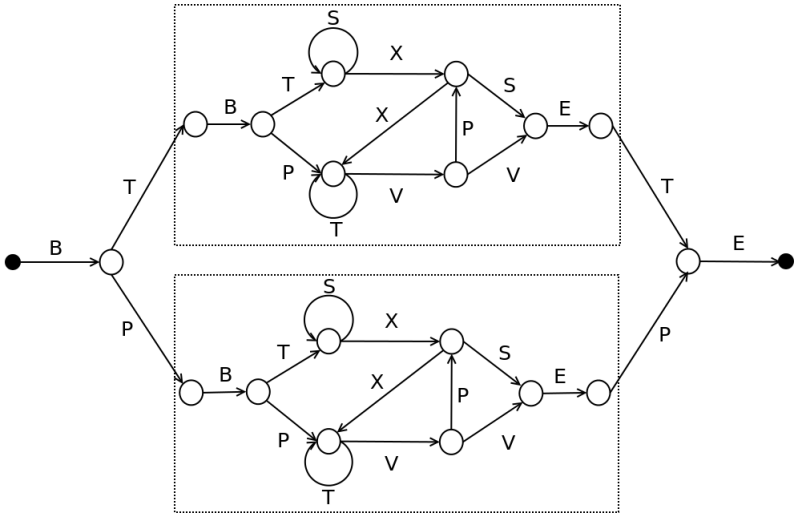
EEL6812 Assignment 3

Instructor: Dr. Shahana Ibrahim
March 25, 2026

1 Task 1: Sequence Classification using Recurrent Models

Consider the scenario in which a string is produced by an Embedded Reber Grammar (ERG), as illustrated in the ERG diagram below. To generate an ERG string, begin at the leftmost black node and randomly select any outgoing edge (in this case, only the edge labelled B is available), noting the letter associated with that edge. Continue this process from the new node until reaching the rightmost black node via the edge labelled E. Thus, BTBTSSXSETE is a valid ERG string. It is important to observe that the sub-networks enclosed in dashed boxes are identical for both the upper and lower branches. These represent the Reber Grammars embedded within this larger grammar. To create longer strings, these dashed boxes can be repeated k times (the E edge inside the dashed box returns to the starting node of the dashed box) before progressing to the T/P edges leading to the terminal node on the right.



For instance, take the string BTBTXSEBTXSEBPVVEBTXXVVETE from this grammar with $k = 4$. If we break it down, the structure becomes clear: BT, BTXSE, BTXSE, BPVVE, BTXXVVE, TE. This structure consists of a prefix (BT), four repeated Reber Grammar strings of varying lengths (BTXSE, BTXSE, BPVVE, BTXXVVE), and a suffix (TE). Determining whether a given string is created by the ERG demands both short-term and long-term memory. Long-term memory involves verifying that the prefix and suffix strings (BT/TE and BP/PE) match. Short-term memory entails monitoring the string for violations in the Reber Grammar transitions (e.g., can a P follow a T?). In this part of the assignment, we will do the implementation of a recurrent neural network associated with these sets of sequences.

1.1 Dataset

The file `Reber.py` outlines a generative process for the ERG and defines a PyTorch Dataset for it. When provided with a dataset size, the code produces a dataset comprising 50% positive samples, 25% negative samples with a random perturbation of one to five characters, and 25% negative samples with the second-to-last letter perturbed. Recognizing these "perturbed" examples necessitates understanding both short- and long-term relationships, as

previously discussed. Note that the examples are converted to a sequence of integer IDs denoting each symbol in the grammar with the `stringToIndex` function.

For training our network, a DataLoader from the dataset is included. However, since the dataset comprises strings of varying lengths, we must provide a custom function to the data loader that specifies how to combine elements from a sampled batch into a single tensor. The `pad_collate` function organizes all batch elements into a $B \times T_{\mathsf{max}}$ tensor for inputs, where T_{max} represents the longest length in the batch. Shorter sequences are padded with zeros. The function returns the resulting input tensor, labels, and a list of lengths.

1.2 Classification Model

The skeleton code provides a definition for a simple recurrent sequence classification model `SimpleRecurrentClassifier`. A correctly implemented model with the default hyperparameters (using the LSTM recurrent network) in `recurrent.py` should achieve validation accuracies in the high 90s.

1.3 Task Objectives

The code `recurrent_neural_network.py` is provided to accomplish the task. You are expected to modify the code, but maintain the class structure. Write a report in pdf (using latex) with the following details:

1. Update the code with missing parts. Implement the `forward` function for `SimpleCurrentClassifier` in `recurrent.py`. Once implemented correctly, it will train your model and output per-epoch training loss and accuracy. Plot the training loss, test loss, training accuracy, and test accuracy. Explain your observations.

   Note that the model definition defines the recurrent network `self.rnn` using an input parameter `recurrent_type`. This helps to choose different rnn models for your task. Implement your forward pass so that it works with `recurrent_type` equal to `nn.RNN, nn.LSTM, or nn.GRU`. See PyTorch documentation for these functions for details.

   Efficiently processing batches with variable input lengths requires some bookkeeping or else the recurrent network will continue to process the padded elements which is ineffient for computation. See `pack_padded_sequence` and `pad_packed_sequence` documentation in PyTorch for the standard way to handle this. Understand their usage and use these functions appropriately.

2. Rerun the experiment and report the result when replacing `nn.LSTM` with `nn.RNN` and `nn.GRU`. Describe your observations. In each scenario, write the dimensions of the learnable parameters of the neural network model.

2 Task 2: Character Generation using Transformer model

In this task, you will train a character-level transformer model to generate text data. The base code is provided (`EEL6812_Transformer_Demo.ipynb`) and please feel free to choose the number of epochs to train accordingly (e.g., using validation loss and until the performance reaches reasonable generation quality). Write a report in pdf using the following details that reflects your understanding about transformer architecture and its training:

- Based on the given implementation, state the dimension of a single training input-label pair (x, y) produced by the dataset class. Then, state the tensor shapes of a batch of inputs and targets for batch size B. Briefly explain why y is a one-step shift of x.
- For a batch input x, explain whether every token position receives a positional encoding. Based on the given implementation, state the dimension of one positional encoding vector and the dimension of the full positional encoding tensor added to a single batch of size B.
- Train the model after removing positional encoding while keeping all other settings unchanged. Compare validation loss curves, final validation loss, at least one attention map where the difference between these scenarios is more pronounced and at least two generated text samples for each scenario. Explain the observed changes and the potential reasons.

- Run at least three values for number of heads in multi-attention mechanism, such as 1, 2, 4, and 8, while keeping the model width d_{model} fixed. Discuss how head count changes attention diversity. Support your answer using visual evidence from attention maps and quantitative evidence using simple statistic such as entropy. To calculate entropy, you may use the following approach.

  The attention map is a matrix $\boldsymbol{A}$ where each ith row of $\boldsymbol{A}$ is a probability distribution over the token that the token i attends to. Since each row sums to 1 after the softmax operation in attention, you can compute the entropy of each row as

  $$H_i = -\sum_j A_{ij} \log A_{ij}$$

  The average over all query positions i can give a summary statistic based on the average entropy.

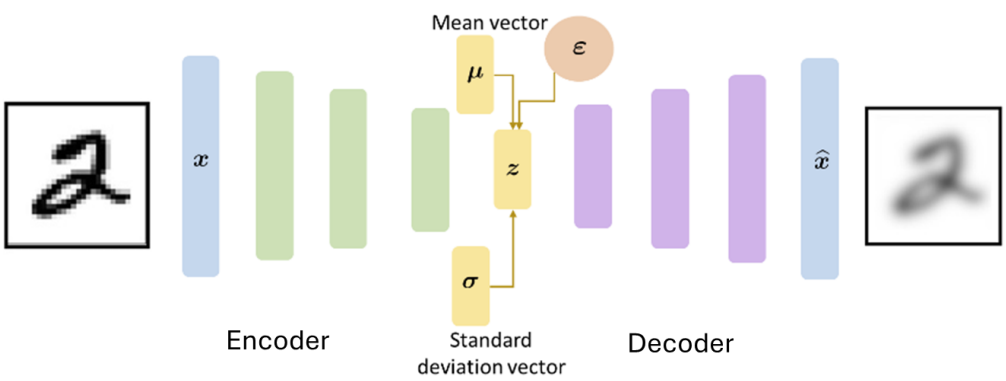
- Run at least three values for number of layers, such as 1, 2, and 4. Discuss whether deeper models produce qualitatively different attention patterns and whether they improve validation loss or text generation. Do you observe any other challenges as depth increases?

- Run at least three context lengths/ block size, such as 32, 64, and 128. Evaluate whether a larger block size improves the ability to capture long-range dependencies. Support this with validation results, generated samples, and at least one example where longer context appears useful. Note that to test long-range dependency, you may have to use a prompt/snippet whose length is larger (e.g., you may use a snippet with 80-120 characters).

- Explain the purpose of causal masking in the transformer language model. To support your explanation, conduct an experiment in which you remove the causal mask while keeping all other hyperparameters fixed. Compare the masked and unmasked models in terms of training loss, validation loss, and generated samples from the same prompt. Discuss the failure mode that arises without masking, and explain why the resulting loss values may be misleading.

3 Task 3: Generative Modeling using Variational Autoencoders

In this task, you will train a Variational Autoencoder (VAE) on the MNIST dataset. Your task is to complete the provided skeleton code `vae.py` by implementing the encoder and decoder components, train the model, and explore the effects of some of the hyperparameters. Below are more specific details about the task objectives:



3.1 Implement the VAE Components

Complete the provided skeleton code for the encoder and decoder networks.

Encoder. Encoder accepts the MNIST input features $\boldsymbol{x}$ and outputs the mean $\boldsymbol{\mu}$ and the standard deviation $\boldsymbol{\sigma}$ of the latent space distribution. For the encoder, you can use a linear layer with 200 hidden units, followed by `ReLU` activation function. After this, you can use two separate linear layers to produce the mean $\boldsymbol{\mu}$ and standard deviation $\boldsymbol{\sigma}$ of the latent space distribution, respectively.
Decoder. The decoder accepts the latent space vector and outputs the reconstructed MNIST data $\widehat{\boldsymbol{x}}$. For the decoder, you can use a linear layer with 200 hidden units that accepts the latent dimension as the input. This will be followed by `ReLU` activation function. The output of this layer will be passed through another linear layer with output dimension same as the dimension of the MNIST data dimension. Finally, you will use a sigmoid activation function.
Latent Vectors. Make sure you use the reparameterization trick to sample latent space representations $\boldsymbol{z}$.

3.2 Train the VAE

Use the binary cross entropy (BCE) loss and the KL divergence (KLD) loss (refer to class lecture notes for the KL divergence loss function definition) to train the VAE model using the MNIST dataset. Plot the training loss function, both BCE and KLD vs epochs. Explain your observations.

3.3 Generate New Samples

Unconditional Generation. After training, generate 10 new samples from randomly sampled latent vectors. Here, the user (you) do not specify any digit for the VAE to generate. You will notice that VAE generates new samples randomly across 10 different digits. This part is already provided in the skeleton code.
Conditional Generation. You can specify the digit to be generated by the VAE. This is known as conditional generation. Given a specific digit, generate new samples by utilizing the encoder's outputs (i.e., mean $\boldsymbol{\mu}$ and standard deviation $\boldsymbol{\sigma}$) for that digit. To accomplish this, sample a latent vector $\boldsymbol{z}$ from the Gaussian distribution defined by $\boldsymbol{\mu}$ and $\boldsymbol{\sigma}$ extracted from the image with the given digit. Then, use the decoder to reconstruct 10 different images. Explain your approach and compare the generated images using unconditional vs conditional generation.

3.4 Modify the Loss Function.

Replace the binary cross-entropy (BCE) loss with mean squared error (MSE) loss. Compare the generated samples and discuss any differences observed between using BCE and MSE loss in terms of generation quality.

3.5 Latent Space Dimension

Modify the latent space dimensionality (e.g., decrease or increase the dimension of $\boldsymbol{z}$). You may use dimensions $\{10, 20, 50\}$ for the study. Observe how this impacts the generation quality and diversity of generated samples. Explain your observations.

Final submission should contain the pdf generated from the latex report and the notebook file (.ipynb file) with the Python code. Zip the files and name "firstname_lastname_a3.zip". Submit the zip folder via Webcourse page. Make sure that the code should run without any edits and produce the outputs you presented in the report.